

Antigravity IDE

Claude Code CLI

Cursor

v0.dev

Expo

Vibe Coding

Cleanly Platform

Build the full cleaning services app
using AI — tools, workflow & prompts

PRIMARY IDE

Antigravity + Claude Code CLI

AI ENGINE

Claude Sonnet (Anthropic)

STACK

Next.js · Expo · Fastify · Neon · Stripe

TIMELINE

Solo: 5–6mo · With team: 2–3mo

Generated March 29, 2026

1 CAN YOU VIBE CODE THIS? HONEST ANSWER

YES

Yes — and you've already done it with Zooli.ai using the same stack (Node.js, Neon, BullMQ, Cloudflare R2, Claude). The patterns transfer directly. Vibe coding will get you 70–80% of the way incredibly fast. The remaining 20% — security, payments, real-time edge cases — needs you to actually understand what Claude is writing. The CLAUDE.md file in your monorepo is what makes this work at scale.

KEY

You describe what to build in plain English. Claude writes the code. You review, test, and steer. You are the product brain + QA + architect. Claude is the senior developer who never sleeps. One CLAUDE.md file per app folder gives Claude persistent memory of your codebase between sessions.

The Three Rules of Successful Vibe Coding

Rule 1 — Commit after every feature

Never let uncommitted work pile up. If Claude breaks something later, you need a clean checkpoint to roll back to. Ask Claude to run `git add + git commit` at the end of every feature.

Rule 2 — Keep CLAUDE.md updated

Every time you add a new table, change a pattern, or make an architectural decision, update CLAUDE.md immediately. This is Claude's long-term memory. Without it, every new session starts from zero.

Rule 3 — Test before building the next thing

Never stack two unbuilt features. Test what Claude just built — click through it, check the network tab, verify the DB record — before asking for the next feature. Bugs compound fast in vibe coding.

2 THE COMPLETE VIBE CODING TOOL STACK

Primary IDEs

Tool	Role	Recommended	Cost	Why for this project
Antigravity IDE	Browser-based AI IDE · Claude already use it on Zooli.ai · same skills system · create	Primary	Included in Claude	You already use it on Zooli.ai · same skills system · create
Claude Code CLI	Terminal agentic coder · sees full	Essential	\$0 (uses Claude API)	Multi-file refactors · runs npm/prisma/git · CLAUDE.md m
Cursor IDE	VS Code fork with AI built-in	Optional	\$20/mo	Best autocomplete if you prefer a local IDE — but Claude
Bolt.new	Full-stack prototyper in browser	Prototyping	\$0 free tier	Great for fast admin panel or dashboard prototype — exp
Lovable.dev	Full-stack app builder	Prototyping	\$0 free tier	Good for company dashboard MVP · generates Next.js y

Design → Code Tools

Tool	What it does	Use for Cleanly
v0.dev (Vercel)	Describe a UI in English → working React + Tailwind code	Customer-web screens: company listing, booking flow, tracking page
Figma → Claude	Share screenshot in Claude Code → Claude recreates HTML screen design from Figma directly into Claude Code	Reusable screen design from Figma directly into Claude Code
Excalidraw	Sketch a rough wireframe → describe to Claude to build	Quick sketch your ideas before prompting Claude to build them

Backend & Database Helpers

Tool / Integration	What Claude can do with it	How to enable
Neon MCP server	Read tables · run queries · inspect schema — all from Claude	Install Neon MCP in Claude Code settings · paste connection
GitHub MCP server	Create PRs · read issues · manage branches from Claude	Install GitHub MCP · authenticate with GitHub token
Stripe + Claude	Claude knows Stripe API deeply · writes full payment integration	Give Claude your Stripe test keys · it runs the integration
Prisma CLI	Claude runs migrations · generates client · inspects DB	npm install -g prisma · Claude calls npx prisma migrate dev
Upstash console	Claude can describe Redis commands · you run in Upstash	No MCP needed · Claude writes the commands · you execu

Mobile (Expo) Workflow with Claude

Step	Tool	How Claude helps
Write screen code	Antigravity / Claude Code	Describe the screen → Claude writes React Native + Tailwind NativeWind code
See it on your phone	Expo Go app	Run npx expo start → scan QR → see changes live in seconds, no build needed
Test GPS + camera	Real device required	Expo Go handles permissions · Claude writes the Expo Location + Camera code
Build for stores	EAS Build (cloud)	Claude writes eas.json config · you run eas build · no Xcode/Android Studio needed
OTA updates	Expo EAS Update	Push JS changes to existing users without App Store review · critical for post-launch fixes

Day 1 Installs — Everything You Need

1. Node.js (required for everything)
Download from: <https://nodejs.org> → LTS version

```
# 2. Claude Code CLI (most important tool)
npm install -g @anthropic-ai/claude-code
claude    ← run this in your project folder to start

# 3. Git
Download from: https://git-scm.com

# 4. Expo CLI + EAS (for mobile apps)
npm install -g @expo/cli eas-cli

# 5. Expo Go on your phone
Search 'Expo Go' in App Store or Google Play → install

# 6. Turborepo (monorepo manager)
npm install -g turbo

# 7. Prisma CLI (database migrations)
npm install -g prisma

# Done. Total install time: ~15 minutes.
```

3 CLAUDE.md — CLAUDE'S LONG-TERM MEMORY

CLAUDE.md is a plain text file you place in each app folder. Every time a new Claude Code session starts, Claude reads it automatically — giving it full context about your project without you having to re-explain anything. This is what separates a productive vibe coding workflow from a frustrating one.

Root CLAUDE.md → /cleanly-monorepo/CLAUDE.md

```
# Cleanly Platform – Claude Context

## What this project is

On-demand mobile cleaning services marketplace for Gulf region.
Washer always comes to the client. Clients choose a company from a list.
Services: car wash, carpet cleaning, sofa cleaning (expandable).
Markets: UAE, Saudi Arabia, Egypt. Languages: Arabic (RTL) + English.

## Monorepo structure

apps/api          → Fastify + Prisma + Socket.io backend
apps/customer-web → Next.js 14 + next-intl + Tailwind (booking)
apps/company-web  → Vite + React + TanStack Query (dashboard)
apps/customer-mobile → Expo + React Native (iOS + Android)
apps/washer-mobile → Expo + React Native (GPS + camera)
apps/admin-web    → Next.js + NextAuth (platform admin)
packages/shared-types → TypeScript interfaces (used everywhere)
packages/i18n     → AR + EN translation strings
prisma/           → Single schema · all 15 tables

## Tech stack

API: Node.js + Fastify + Prisma ORM + Zod validation
DB:  Neon PostgreSQL + PostGIS + Upstash Redis
Auth: JWT 15min access + 30d refresh + Twilio OTP (customers)
Payments: Stripe Payments + Stripe Connect (company payouts)
Real-time: Socket.io · rooms per order for GPS tracking
Storage: Cloudflare R2 · signed URLs only · never public
Push: Expo Push (FCM + APNs) · SMS: Twilio · Email: Resend
Deploy: Vercel (frontends) + Railway (API + workers)

## Rules you must ALWAYS follow

- Every API endpoint must have Zod validation on body + params
- All DB queries through Prisma – no raw SQL from user input
- All UI strings use i18n keys – never hardcode English text
- Content fields have _en and _ar variants (name_en, name_ar)
- JWT role checked on every protected route
- Never log phone numbers, card data, or tokens
- Photos always go to R2 – never stored in DB
- Rate limiting required on OTP, payment, and order endpoints
- RTL layout must work in Arabic – use CSS logical properties
```

Order status states

confirmed → assigned → en_route → arrived → in_progress → completed
(cancelled and disputed are terminal states)

When I say 'add a screen'

Build for customer-web AND customer-mobile unless stated otherwise.
Both must support AR + EN with full RTL layout in Arabic mode.

Per-App CLAUDE.md Example → apps/api/CLAUDE.md

API App – Claude Context

This app

Fastify REST API + Socket.io · runs on Railway · port 3001
All routes prefixed /v1/ · JSON responses only

File structure convention

src/routes/ → one file per domain (auth, orders, companies...)
src/middleware/ → auth.ts, rateLimit.ts, validate.ts
src/services/ → business logic (never in route handlers)
src/workers/ → BullMQ job processors (email, payouts)
src/lib/ → db.ts (Prisma), redis.ts, stripe.ts, socket.ts

Route pattern – always follow this

1. Zod schema validates the request
 2. Auth middleware verifies JWT + checks role
 3. Service layer executes business logic
 4. Route handler calls service, returns typed response
- Never put business logic directly in route handlers.

Env vars available

DATABASE_URL, REDIS_URL, JWT_SECRET, JWT_REFRESH_SECRET
STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET
TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_SERVICE_SID
CLOUDFLARE_R2_BUCKET, CLOUDFLARE_R2_ACCESS_KEY, CLOUDFLARE_R2_SECRET
RESEND_API_KEY, EXPO_ACCESS_TOKEN

Update CLAUDE.md every time you:

Add a new DB table · Change a naming convention · Add a new env variable ·
Make an architectural decision · Add a new package or third-party service ·
Change a route prefix or file structure

4 PROMPT PLAYBOOK — HOW TO TALK TO CLAUDE

The Golden Rule: Give Claude the spec, not just the name.

Claude writes much better code when you give it exact fields, expected behavior, edge cases, and which files to touch. Vague prompts produce vague code that needs 3 rounds of correction. Specific prompts produce production-ready code on the first try.

Building a new API endpoint

GOOD — specific and complete

```
Build the company listing endpoint:
GET /v1/companies
Query params: city_id (required), category_id (required), sort (optional:
rating|price|eta)
Return only companies where is_active=true AND is_verified=true
Join company_cities + company_categories tables
Include avg_rating, total_reviews, price_from (min package price), estimated
ETA
Sort by avg_rating DESC by default
Zod validation on all params · Rate limit: 30 req/min
Cache result in Redis by city_id+category_id with 5min TTL
```

BAD — too vague

"Build the company listing" — Claude will guess the fields, skip caching, miss the rate limit.

Building a UI screen

GOOD — full requirements

```
Build the company listing screen in apps/customer-web:
Show companies as cards: name, avg_rating (stars), price_from, eta_minutes
Show service category tags on each card (car wash / carpet / sofa)
Filter pills at top: All · Top rated · Fastest · Price low-high
Fetch from GET /v1/companies using TanStack Query
Show skeleton loading cards while fetching
Show empty state if no companies for this city
Fully RTL mirrored when Arabic is active (use CSS logical properties)
All text through i18n keys from packages/i18n — no hardcoded strings
```

BAD — missing context

"Make a company list screen" — Claude skips RTL, hardcodes English, misses loading state.

Database schema change

**GOOD — exact
field definitions**

```
Add a promo_codes table to prisma/schema.prisma:
id String @id @default(uuid())
code String @unique
type PromoType (enum: PERCENT, FIXED)
value Float
max_uses Int
used_count Int @default(0)
category_id String? (nullable FK to service_categories)
city_id String? (nullable FK to cities)
expires_at DateTime?
created_at DateTime @default(now())
Then: run migration · update shared-types PromoCode interface
```

**BAD — no field
detail**

"Add promo codes" — Claude invents an incompatible schema that breaks your conventions.

Debugging an issue

**GOOD — full
context + error**

The Socket.io GPS tracking stops sending updates to the customer after the washer marks "arrived". The washer is still broadcasting location. Here is the error in the customer app console: [paste error]. Here is the Socket.io room join code: [paste code]. Here is the washer emit code: [paste code]. Find the bug and fix it.

**BAD — no
context**

"GPS tracking is broken, fix it" — Claude has nothing to work with and will guess randomly.

Asking for tests

**GOOD — specify
cases**

```
Write Vitest unit tests for src/services/orderService.ts createOrder()
function. Test these cases:
1. Successful order creation with valid data
2. Company not serving this city (should throw 400)
3. Package not belonging to chosen company (should throw 400)
4. Customer wallet insufficient for wallet payment (should throw 402)
5. Stripe PaymentIntent creation failure (should throw 500 + log to Sentry)
Mock: Prisma, Stripe, Redis. Use vi.mock().
```

**BAD — no test
cases defined**

"Write tests for the order service" — Claude writes 2 happy-path tests and nothing useful.

5 THE DAILY VIBE CODING WORKFLOW

1 Open Claude Code in your project folder
cd cleanly-monorepo → claude (Claude reads CLAUDE.md automatically) → say what you're building today

2 Give Claude the full feature spec
Not just the name — the exact fields, behavior, edge cases, files to touch. Copy from the blueprint PDF.

3 Claude writes code — you steer
Watch what Claude builds. If it goes wrong, say 'stop — do it this way instead' immediately. Don't let it compound a mistake.

4 Test before moving on
Run the app. Click through the flow. Check the DB record. Check the network tab. Fix any issues before the next feature.

5 Ask Claude to write tests
'Write Vitest tests for what we just built, covering: [list the cases]' — do this every time, not at the end.

6 Commit everything
'Commit all changes with message: feat(api): add order creation with city validation' — Claude runs git add + commit + push.

7 Use /compact or /clear when needed
/compact summarizes history to free context. /clear starts fresh (Claude re-reads CLAUDE.md). Switch features between sessions.

Claude Code Commands to Know

Command	When to use
claude	Start an interactive session in the current folder · Claude reads CLAUDE.md first
claude 'do X'	One-shot command without interactive mode · good for small tasks like running migrations
claude --resume	Resume the last session · useful when you close terminal and come back
/compact	Inside a session: summarize history to free up context window · use when Claude gets forgetful
/clear	Inside a session: start fresh · Claude re-reads CLAUDE.md · use when switching features
claude --model	Switch between Claude models · Sonnet for most tasks · Opus for complex architecture decisions

Session Structure — One Feature Per Session

Start of session	"Today we are building the washer GPS tracking feature. The washer mobile app should emit location every
-------------------------	--

During session	Let Claude build the full feature. Ask clarifying questions as they come up. Test after each file is written if possible.
End of session	"Before we finish: write Vitest tests for the socket service. Then commit everything with message: feat: washer." Then commit everything with message: feat: washer.
Next session	Start with a new /clear → new CLAUDE.md read → next feature. Never carry over half-finished work.

6 WHAT CLAUDE HANDLES VS WHAT YOU HANDLE

Claude does great — trust it

- All boilerplate code (routes, components, schemas)
- Stripe payment integration (knows Stripe API deeply)
- Zod validation schemas for every endpoint
- Prisma queries: joins, filters, aggregations
- React + Tailwind UI screens
- RTL layout with CSS logical properties
- i18n setup: next-intl, translation files, RTL detection
- Socket.io room setup and GPS broadcast pattern
- BullMQ job processors: email, payout queues
- Error handling: try/catch, HTTP status codes, Sentry
- Vitest unit tests when you describe the cases
- Playwright E2E test scripts
- Git operations: add, commit, push, branch
- Large refactors: split files, extract services, rename
- npm install, prisma migrate, turbo run
- GitHub Actions CI pipeline YAML
- Dockerfile and Railway/Vercel config

Review carefully before shipping

- JWT implementation: check expiry + refresh + role guards
- Stripe webhook: verify signature yourself — real money
- Rate limiting: check numbers make sense for your traffic
- Socket.io rooms: verify only right users join right rooms
- R2 signed URLs: confirm photos are NOT public by default
- Stripe Connect payouts: test commission split math in test mode
- GPS plausibility checks: fraud detection actually running
- Arabic RTL: test on real device or browser in Arabic mode
- Database migrations: read SQL before running on production
- Admin role guards: verify admin endpoints are truly restricted
- Env vars: ensure secrets are never logged or exposed

You must do this yourself

- Test with real Stripe money before going live
- Test GPS + camera on real physical device
- Talk to 5 car wash companies — Claude can't do sales
- Product decisions: pricing, commission, feature priority
- Legal: UAE trade license, terms of service, privacy policy
- App Store screenshots, descriptions, review responses
- Arabic translation: get native speaker to review all AR strings
- Security pen test before public launch
- Company onboarding: train them on the dashboard
- Customer support: handle complaints and disputes
- Hiring: Claude can't interview your washer recruits

7 FEATURE-BY-FEATURE BUILD ORDER WITH PROMPTS

Build in this exact order. Each feature depends on the previous one. Do not skip ahead — if you try to build the tracking screen before auth exists, Claude will invent auth and you will have to redo it.

Week 1 — Foundation

Monorepo scaffold	<i>Prompt:</i> npx create-turbo@latest cleanly-monorepo, then ask Claude: 'Set up the Turborepo workspace with 6 apps (api, customer-web, company-web, customer-mobile, washer-mobile, admin-web) and 3 packages (shared-types, i18n, ui-components). Add TypeScript, ESLint, and Prettier configs to each. Create the root CLAUDE.md.'
Prisma schema + DB	<i>Prompt:</i> 'Create prisma/schema.prisma with all 15 tables from [paste schema]. Then run the first migration on Neon dev branch. Then seed: UAE cities, 3 service categories (car_wash, carpet, sofa), 1 test company, 2 test packages.'
Fastify API scaffold	<i>Prompt:</i> 'Set up apps/api with Fastify, Prisma client, JWT plugin, Zod, Helmet, CORS. Create the folder structure: routes/, services/, middleware/, lib/. Add health endpoint GET /health that returns {status:ok, db:ok}.'

Week 2 — Auth (all roles)

Customer OTP auth	<i>Prompt:</i> 'Build phone OTP auth: POST /v1/auth/send-otp (Twilio Verify, rate limit 3/phone/15min) and POST /v1/auth/verify-otp (returns JWT access + refresh tokens). Tokens stored in HttpOnly cookies on web, SecureStore on mobile. Zod validation on both endpoints.'
Company auth	<i>Prompt:</i> 'Build company auth: POST /v1/company/auth/login (email + bcrypt password). JWT payload includes role:company_admin + company_id. Middleware: requireCompanyAuth that validates this role.'
Washer auth	<i>Prompt:</i> 'Build washer auth using same Twilio OTP flow as customer but JWT payload includes role:washer + company_id + washer_id. Middleware: requireWasherAuth.'

Week 3 — Company onboarding

Company profile setup	<i>Prompt:</i> 'Build company onboarding endpoints: POST /v1/company/profile (saves name_en, name_ar, description_en, description_ar, logo_url). POST /v1/company/cities (saves city_id array to company_cities table). POST /v1/company/categories (saves category slugs to company_categories).'
Package + add-on management	<i>Prompt:</i> 'Build CRUD for packages: GET/POST/PUT/DELETE /v1/company/packages. Fields: name_en, name_ar, description_en, description_ar, base_price, unit_type (per_vehicle/per_room/per_seat), duration_minutes, category_id. Same CRUD pattern for /v1/company/addons.'
Company dashboard UI	<i>Prompt:</i> 'Build apps/company-web: sidebar nav (Orders, Packages, Staff, Analytics, Settings). Start with Settings: profile form (EN + AR fields), city coverage multi-select, category toggles, package list with add/edit/delete. Use TanStack Query. Bilingual: all labels through i18n.'

Week 4 — Customer booking flow

Company listing API	<i>Prompt:</i> 'Build GET /v1/companies?city_id=X&category_id=Y. Filter: is_active+is_verified+city coverage+category. Return: id, name_en, name_ar, avg_rating, total_reviews, price_from, logo_url. Cache in Redis by city_id+category_id, TTL 5min. Rate limit 30/min.'
Customer web — home + listing	<i>Prompt:</i> 'Build customer-web home screen: language selector (AR/EN), service category picker (3 cards), GPS city detection with manual fallback. Company listing screen: cards with rating + price + ETA, filter pills (All/Top rated/Fastest). Full RTL layout in Arabic. All text through next-intl i18n keys.'
Company profile + package selection	<i>Prompt:</i> 'Build customer-web company profile screen: hero (logo, name, rating, reviews count), tab: Packages (list with price + duration + unit type), tab: Reviews. Quantity selector for carpet (per room) and sofa (per seat) packages. Add-ons section with checkboxes. Running total at bottom. Book button.'

Week 5 — Payments + Order creation

Stripe payment flow	<i>Prompt:</i> 'Build POST /v1/orders: creates order row (status=confirmed), creates Stripe PaymentIntent for total amount, returns client_secret. Build POST /v1/orders/:id/confirm-payment: verifies PaymentIntent succeeded, saves payment row, triggers FCM push to company. Build Stripe webhook handler at POST /v1/webhooks/stripe with signature verification.'
Checkout UI	<i>Prompt:</i> 'Build checkout screen: order summary (package + addons + fee + total), payment method selector (saved card / Apple Pay / wallet credits), Stripe Elements card input, confirm button. Success screen with order ID and tracking link.'

Week 6 — Live tracking (the wow feature)

Socket.io GPS system	<i>Prompt:</i> 'Build Socket.io server in apps/api. Events: washer:join-order (washer joins room order:{id}), customer:join-order (customer joins same room), washer:location (emits lat/lng every 5s). Store last location in Redis key washer:{id} TTL 10s. Emit order:status-update to room when status changes.'
Washer mobile — navigation	<i>Prompt:</i> 'Build washer app: job alert screen (accept/decline + 30s countdown), active job screen (customer name + location + parking note + Google Maps deep link button), GPS background tracking with Expo Location that emits to Socket.io every 5s, Arrived button, checklist screen per service category, photo capture (Expo Camera) + R2 upload, Done button that calls PATCH /v1/orders/:id/complete.'
Customer tracking screen	<i>Prompt:</i> 'Build customer-web order tracking screen: 5-step progress bar (confirmed/en_route/arrived/in_progress/done), Mapbox map with washer dot that moves in real time via Socket.io, ETA countdown, company name + washer name + rating, before photo shown when washer arrives, after photo shown when done.'

8 TIMELINE & COST ESTIMATE

Timeline by Team Size

Scenario	To Private Beta	To Public Launch	To Regional Scale	Monthly Cost
Solo (you + Claude only) 4–6 hrs/day	~10 weeks	~20 weeks	~30 weeks	Claude Pro \$20 + infra \$50
You + 1 developer + Claude Each owns 2–3 surfaces	~6 weeks	~12 weeks	~20 weeks	Claude Pro \$40 + infra \$100
Full team (4 people) + Claude Each person owns one surface	~3 weeks	~7 weeks	~14 weeks	Claude Pro \$80 + infra \$200

Claude Subscription You Need

Plan	Price	Best for	Why
Claude Pro	\$20/mo	Solo vibe coder	Enough for most sessions · access to Sonnet + Opus · Claude Code included
Claude Pro (×2)	\$40/mo	You + 1 dev	One account each · separate context and CLAUDE.md per developer
Claude for Teams	\$30/user/mo	Team of 4+	Shared billing · higher rate limits · better for intensive daily building

Infrastructure Cost — Vibe Coding Friendly (Pay As You Grow)

Service	Free / Starter	When to upgrade	Trigger
Vercel	Free (hobby)	\$20/mo Pro	Custom domain + preview deployments needed
Railway	\$5/mo starter	\$20/mo+	API getting real traffic (>500 req/day)
Neon DB	Free (0.5GB)	\$19/mo Launch	DB > 500MB or need staging branch
Upstash Redis	Free (10K/day)	\$10/mo Pay-as-go	Rate limit cache + Socket.io adapter needed
Cloudflare R2	Free (10GB)	Pay as you grow	Photo storage grows beyond 10GB
Twilio	Trial \$15 credit	Pay-as-you-go	Real users sending OTPs (~\$0.05/SMS)
Stripe	No monthly fee	2.9% + 30¢ per txn	Day 1 — just pay per transaction
Resend	Free (100/day)	\$20/mo	Email volume > 100/day
Expo EAS	Free (30 builds/mo)	\$29/mo	Building mobile app more than 30×/month
TOTAL LAUNCH	~\$20–50/mo	~\$200–400/mo	When you hit real user volume

Week 1 — Do These First (Before Writing Any Code)

Create accounts (30 min each)

- GitHub organization → create monorepo
- Vercel account → 4 projects: customer-web, company-web, admin-web, docs
- Railway account → 1 project (API + worker services)
- Neon account → 1 project → 3 branches: dev, staging, prod
- Upstash account → 1 Redis database (dev) + 1 (prod)

- Cloudflare account → 1 R2 bucket: cleanly-photos
- Stripe account → enable Connect → save test API keys
- Twilio account → buy UAE number → enable Verify service
- Resend account → verify your sending domain
- Expo account → EAS CLI login
- Mapbox account → create access token
- Sentry account → 5 projects (one per app surface)

Submit immediately — these take weeks

- Apple Developer account (\$99/yr) — takes 1–3 days to verify
- Google Play Console (\$25 one-time) — usually instant
- 360dialog WhatsApp Business verification — takes 2–3 weeks
- TikTok API app review (if needed) — 2–4 weeks
- UAE trade license (if registering a company) — 2–4 weeks

First Claude Code session

- Run: `npm install -g @anthropic-ai/claude-code`
- Create cleanly-monorepo folder
- `cd cleanly-monorepo → claude`
- Say: 'Create the root CLAUDE.md with [paste content from Section 3]'
- Say: 'Now scaffold the Turborepo monorepo with 6 apps and 3 packages'
- Say: 'Now create the Prisma schema with all 15 tables'
- Say: 'Now run the first migration on the Neon dev branch'
- Say: 'Commit everything: chore: monorepo scaffold + prisma schema'